

Παράλληλος και Κατανεμημένος Υπολογισμός I

Κατανεμημένη Μνήμη Μεταβίβαση Μηνυμάτων (συν. 3)

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2024



Συντονιστής εργαζόμενος
Master worker



Πελάτης διακομιστής Συντονιστής εργαζόμενος

- Πελάτης διακομιστής: Συνήθως ο διακομιστής εκτελείται συνεχώς, σε υπολογιστικό σύστημα με μεγαλύτερη ισχύ και με πρόσβαση στα διατηρούμενα δεδομένα. Οι πελάτες ζητούν από τον διακομιστή να εκτελέσει απαιτητικούς υπολογισμούς ή/και να αποθηκεύσει/ανακτήσει δεδομένα.
- Συντονιστής εργαζόμενος: Συνήθως ο συντονιστής είναι απλά διεκπεραιωτής αιτημάτων και συλλέκτης αποτελεσμάτων. Οι εργαζόμενοι εκτελούν τους υπολογισμούς που ζητά ο συντονιστής και επιστρέφουν τα αποτελέσματα. Έχουν πρόσβαση σε τοπικά ή κατανεμημένα δεδομένα.

Συντονιστής και εργαζόμενος: μοντέλο υπολογισμού 1

- Εργαζόμενος

μια διεργασία που αναμένει αιτήματα (εργασίες), τις οποίες εκτελεί και επιστρέφει τα αποτελέσματα στον αιτούντα.

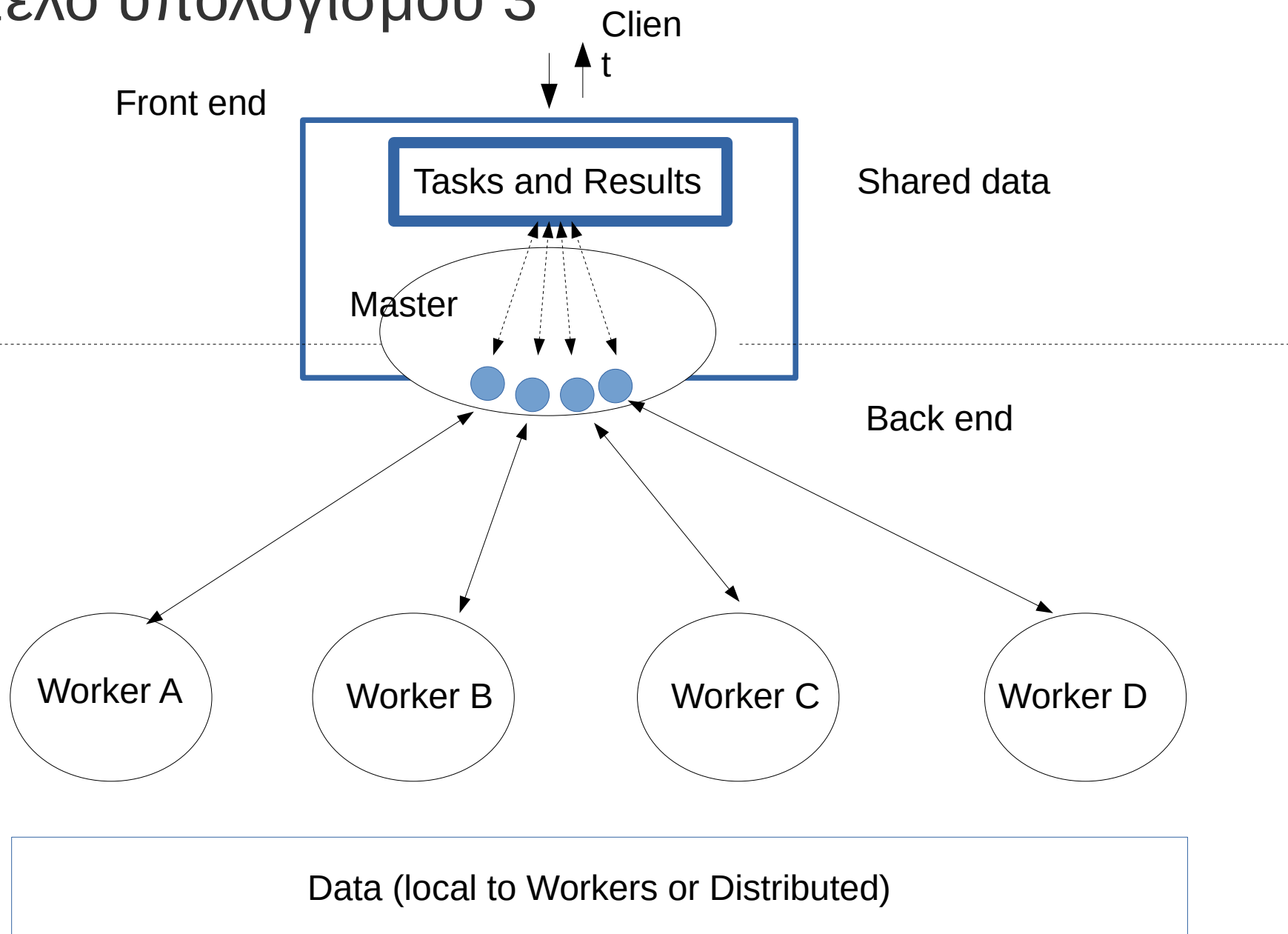
- Συντονιστής

μια διεργασία που αποστέλλει αιτήματα σε πολλούς (συνήθως) εργαζομένους, διανέμοντας έτσι μια μεγάλη εργασία σε πολλές μικρότερες, και συλλέγει τα απαντήσεις των εργαζομένων για να συνθέσει το συνολικό αποτέλεσμα.

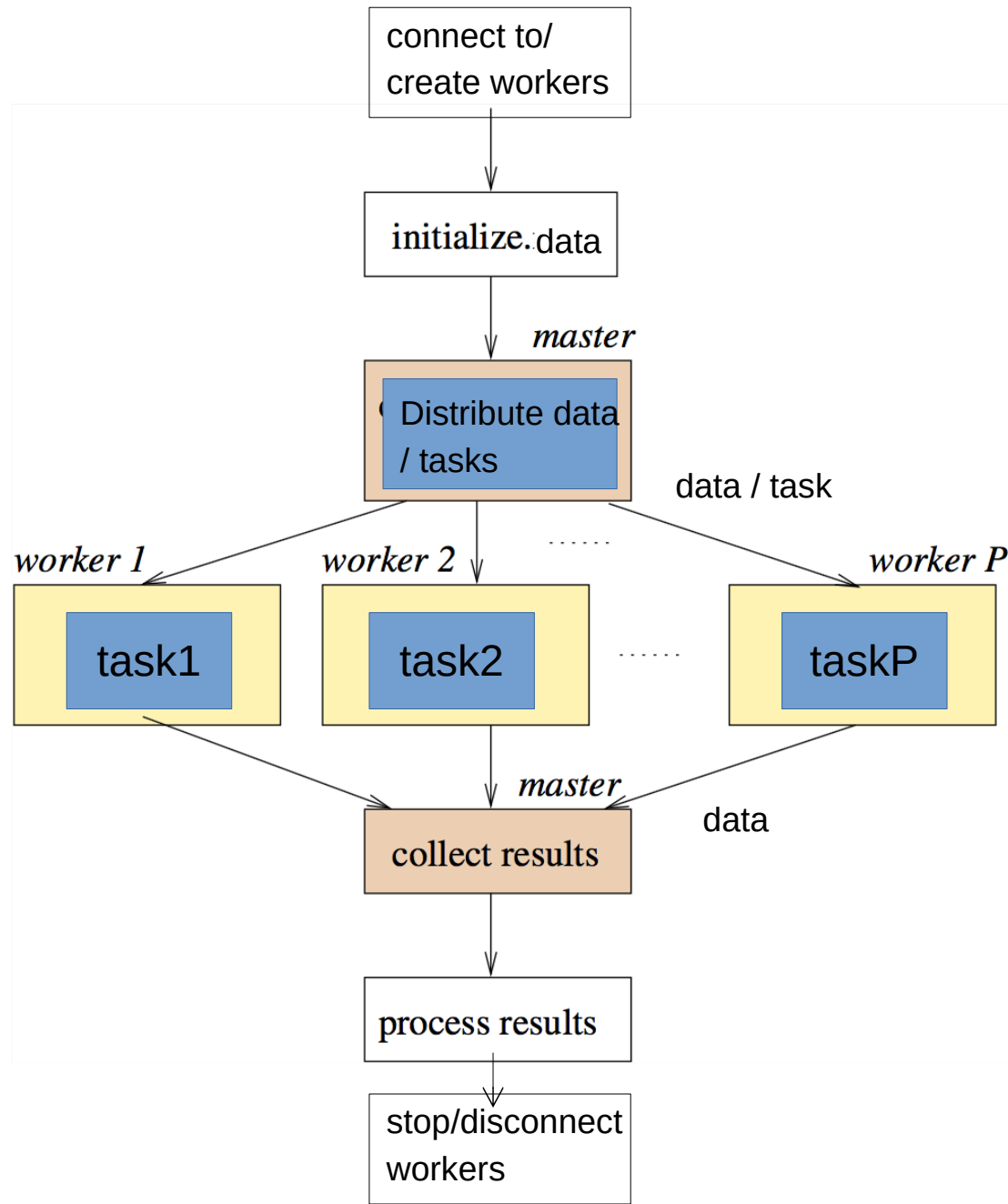
Συντονιστής και εργαζόμενος: μοντέλο υπολογισμού 2

- Μοιάζει σαν αντιστροφή του Πελάτη Διακομιστή
- Μεγάλη χρήση σε κέντρα δεδομένων, συστοιχίες υπολογιστών για την διανομή υπολογισμών και συλλογή αποτελεσμάτων.
- Συνήθως “κλειστά” συστήματα Συντονιστή Εργαζομένων: οι Εργαζόμενοι δεν είναι προσβάσιμοι παρά μόνο μέσω του Συντονιστή. Ο Συντονιστής είναι το “front-end” και οι Εργαζόμενοι το “back-end”.

Συντονιστής και εργαζόμενος: μοντέλο υπολογισμού 3



Συντονιστής και εργαζόμενος: μοντέλο υπολογισμού 4



Grouping
Load Balancing

Συντονιστής και εργαζόμενος

Fork-Join και SPMD

- Η δομή της εφαρμογής μοιάζει με τη δομή Fork-Join των παράλληλων εφαρμογών (με νήματα ή διεργασίες) σε μοιραζόμενη μνήμη.
- Ο κώδικας των νημάτων και των πελατών συνήθως ακολουθεί τη μέθοδο SPMD: οι πελάτες έχουν id και αναλαμβάνουν έργο με βάση το id τους.
- Διαφορές:
- Η δημιουργία των πελατών είναι συνήθως στατική: δημιουργούνται στην αρχή της εφαρμογής και έχουν διάρκεια ζωής όσο η εφαρμογή.
- Τα νήματα συνήθως απλά επικοινωνούν με τους πελάτες, δεν εκτελούν σύνθετους υπολογισμούς.



Συντονιστής και εργαζόμενος

Map (Filter) Reduce

Διανομή Υπολογισμός Συλλογή

- Απεικόνιση : διανομή εργασίας και δεδομένων από τον συντονιστή στους εργαζόμενους μέσω των νημάτων του συντονιστή.
- Απεικόνιση (Φιλτράρισμα) : εκτέλεση υπολογισμών στα τοπικά δεδομένα στους εργαζόμενους.
- Αναγωγή: Συλλογή δεδομένων στον συντονιστή, με αποστολή από τους εργαζόμενους στον συντονιστή μέσω των νημάτων του συντονιστή.

```
//shared type/class definition
struct/class shared {
    lock_type lock;
    int[] tasks; //δεδομένα
    int[] results; //αποτελέσματα

}
```

Συντονιστής
εργαζόμενος
περίγραμμα 1

```
// main
MasterTCP
{
    my_struct = new shared (my_lock, my_data);
    create ServerSocket(Port); //bind, listen

    for (i=0; i<numberOfWorkers; i++) {
        connection = accept (ServerSocket);
        dispatch masterThread[i] (connection, my_struct);
    }

    for (i=0; i<numberOfWorkers; i++) {
        wait for masterThread[i] to finish (join)
    }

    produce result (my_struct)
}
```

Συντονιστής εργαζόμενος περίγραμμα 2

```
MasterProtocol (shared my_struct)
```

```
{  
    ...    concurrent access to my_struct  
}
```

```
MasterThread (my_connection, shared my_struct)
```

```
{  
    //my_connection establishes inStream and outStream  
  
    messageToWorker =  
        MasterProtocol.PrepareMessage(my_ref);  
    Send/Write (messageToWorker, out);  
    Receive/Read (messageFromWorker, in);  
    MasterProtocol.GetResult(messageFromWorker, my_struct);  
    Close Socket();  
}
```

Συντονιστής εργαζόμενος περίγραμμα 3

```
WorkerProtocol { ... }
```

```
Worker
```

```
{
```

```
    Socket();
```

```
    connection = Connect (Server);
```

```
    Receive/Read (messageFromMaster, in);
```

```
    messageToMaster =
```

```
        WorkerProtocol.Process(messageFromMaster);
```

```
    Send/Write (messageToMaster, out);
```

```
    Close Socket();
```

```
}
```

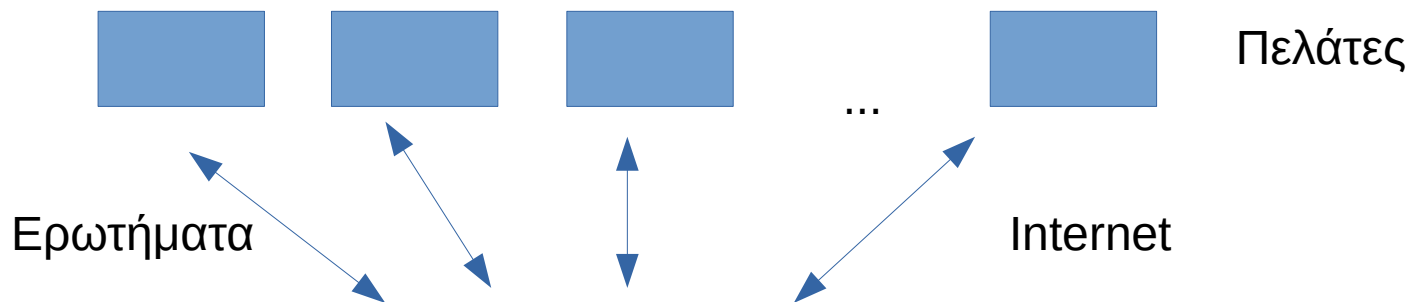
Συντονιστής και εργαζόμενος

Διανομή εργασιών και δεδομένων, συλλογή αποτελεσμάτων

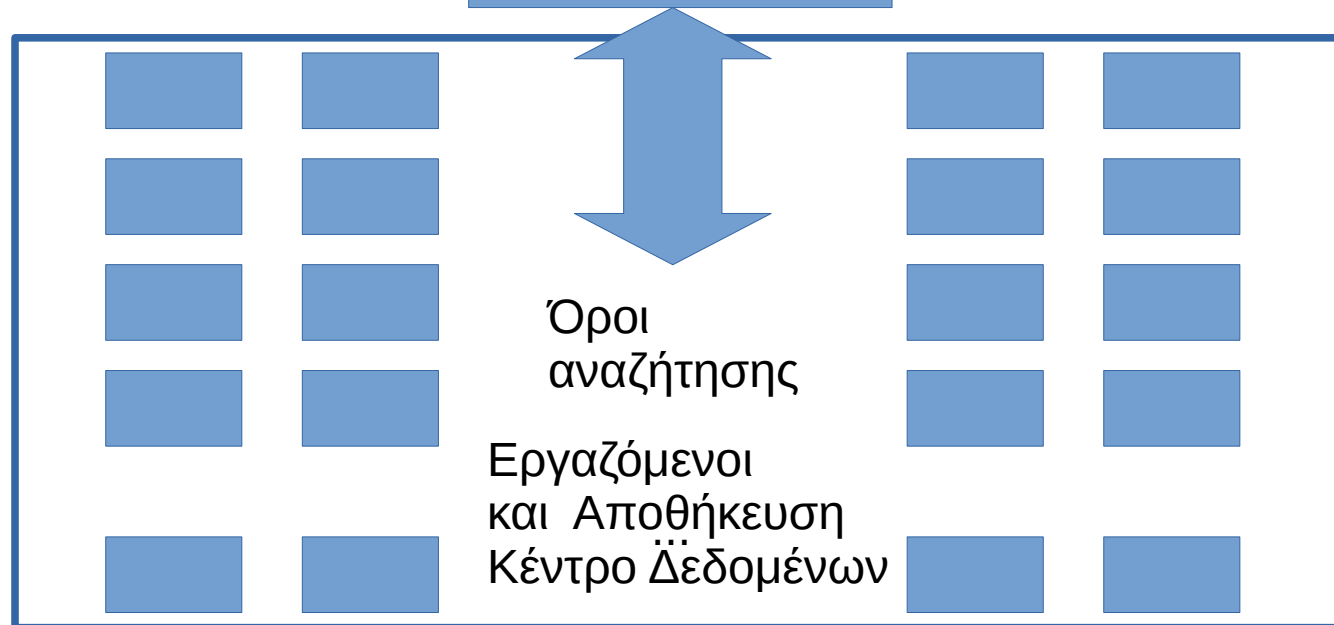
- Κατά την απεικόνιση, η κατανομή εργασιών και δεδομένων μπορεί να είναι στατική, κυκλική, δυναμική, δεξαμενή εργασιών..
- Κατά την αναγωγή η συλλογή των αποτελεσμάτων δεδομένων μπορεί να λάβει όποια μορφή αναγωγής επιλέξουμε, πχ με αμοιβαίο αποκλεισμό ή με ακολουθιακά (με πίνακα ή στο join).
- Γενικά ισχύουν τα πρότυπα που έχουμε περιγράψει στη μοιραζόμενη μνήμη, με εξαίρεση τον αναδρομικό παραλληλισμό.

Παράδειγμα: απλή μηχανή αναζήτησης 1

Κατανεμημένος
Υπολογισμός



Ταυτόχρονος
Υπολογισμός



Παράλληλος
Υπολογισμός

Πολλαπλά επίπεδα Παράλληλου και Κατανεμημένου υπολογισμού.

Παράδειγμα: απλή μηχανή αναζήτησης 2

- Πελάτες: περιηγητές/εφαρμογές που στέλνουν ερωτήματα στη μηχανή αναζήτησης και περιμένουν απαντήσεις: Κατανεμημένος υπολογισμός, πελάτης-διακομιστής.
- Διακομιστής: δέχεται τα ερωτήματα πολλών πελατών και τους εξυπηρετεί ταυτόχρονα: Ταυτόχρονος υπολογισμός, πολυνηματικός διακομιστής, ένα νήμα ανά πελάτη (απλή λύση).
- Για κάθε νήμα του διακομιστή δημιουργείται ένα ξεχωριστό στιγμιότυπο Συντονιστή – Εργαζομένων για την εξυπηρέτηση του ερωτήματος του συγκεκριμένου πελάτη.

Παράδειγμα: απλή μηχανή αναζήτησης 3

- Συντονιστής: ο συντονιστής διανέμει κάθε ερώτημα σε πολλούς υπολογιστές (εργαζόμενους) στο κέντρο δεδομένων, οι οποίοι επιστρέφουν τις επιμέρους απαντήσεις. Ο συντονιστής συλλέγει και κατατάσσει τις απαντήσεις για κάθε ερώτημα: Παράλληλος και Κατανεμημένος υπολογισμός.
- Εργαζόμενοι: υπολογιστές σε κέντρο δεδομένων συνδεδεμένοι με ταχύ δίκτυο, εκτελούν την ίδια εργασία ταυτόχρονα αλλά σε διαφορετικά δεδομένα, τα οποία είναι αποθηκευμένα τοπικά στους εργαζόμενους ή σε ξεχωριστά συστήματα αποθήκευσης δεδομένων: Παράλληλος και Κατανεμημένος υπολογισμός

Παράδειγμα: απλή μηχανή αναζήτησης 4

- Ο κάθε εργαζόμενος συνήθως συνήθως είναι σύστημα με πολλαπλούς επεξεργαστές και επιταχυντές, έτσι ώστε έχουμε επιπλέον παραλληλισμό στους υπολογισμούς σε κάθε εργαζόμενο.
- Η ταυτόχρονη εκτέλεση πολλών στιγμιοτύπων συντονιστή – εργαζομένων απαιτεί διαχείριση: μια πιθανή λύση είναι οργάνωση των εργαζομένων σε ομάδες ώστε κάθε ομάδα να συνεργάζεται με διαφορετικό συντονιστή.
- Υπάρχουν ειδικά συστήματα για την εξισορρόπηση των υπολογισμών συντονιστή – εργαζομένων σε κέντρα δεδομένων.

Παράδειγμα: απλή μηχανή αναζήτησης 4

- Η συγκέντρωση των υπολογισμών σε έναν πολυνηματικό διακομιστή και έναν (τον ίδιο;) συντονιστή μπορεί να προκαλέσει συμφόρηση και είναι ευαίσθητη σε πιθανή κατάρρευση.
- Λύση: χρήση συστημάτων πολλαπλών διακομιστών που επιτυγχάνουν εξισορρόπηση φορτίου τόσο στα εισεργόμενα αιτήματα (εξυπηρέτηση πελατών) όσο και στα πολλαπλά στιγμιότυπα συντονιστών.
- Αντίστοιχα μπορούμε να αναπτύξουμε και τα συστήματα συλλογής και επεξεργασίας ιστοσελίδων για την προετοιμασία των δεδομένων της μηχανής αναζήτησης.

Συντονιστής Εργαζόμενος: υλοποιήσεις

- Clusters υψηλών επιδόσεων

Message Passing Interface (MPI)

- Clusters εξισορρόπησης φορτίου ή/και υψηλής διαθεσιμότητας (πχ HAProxy)

- Data Centers:

Clouds, Big Data, Search Engines, Social Networks

Apache Hadoop: MapReduce, HDFS, YARN

Apache Spark

- Grids

- Εθελοντικός Υπολογισμός

BOINC



Serializable objects

TCP Server: serializable objects 1

```
import java.io.*;
import java.net.*;

public class Server {
    private static final int PORT = 1234;

    public static void main(String[] arg)
        throws IOException, ClassNotFoundException {

        ServerSocket socketConnection = new ServerSocket(PORT);
        System.out.println("Server Waiting");

        Socket pipe = socketConnection.accept();

        ObjectInputStream serverInputStream = new
            ObjectInputStream(pipe.getInputStream());
        ObjectOutputStream serverOutputStream = new
            ObjectOutputStream(pipe.getOutputStream());

        ...
    }
}
```

TCP Server: serializable objects 2

...

```
ServerProtocol serverProt = new ServerProtocol();
```

```
Request req = (Request) serverInputStream.readObject();
```

```
Reply rep = serverProt.processRequest(req);
```

```
serverOutputStream.writeObject(rep);
```

```
serverInputStream.close();
```

```
serverOutputStream.close();
```

```
pipe.close();
```

```
}
```

```
}
```

Code/Simple-client-server-tcp-serializable

TCP Client: serializable objects 1

```
import java.io.*;
import java.net.*;

public class Client {
    private static final String HOST = "localhost";
    private static final int PORT = 1234;

    public static void main(String[] arg)
        throws IOException, ClassNotFoundException {

        Socket socketConnection = new Socket(HOST, PORT);

        ObjectOutputStream clientOutputStream = new
            ObjectOutputStream(socketConnection.getOutputStream());
        ObjectInputStream clientInputStream = new
            ObjectInputStream(socketConnection.getInputStream());

        ...
    }
}
```

TCP Client: serializable objects 2

...

```
Request req = new Request("MUL", 5, 6);
```

```
clientOutputStream.writeObject(req);
```

```
Reply rep = (Reply)clientInputStream.readObject();
```

```
System.out.println("Reply opcode = " + rep .getOpcode());
```

```
System.out.println("Reply value = " + rep .getValue());
```

```
clientOutputStream.close();
```

```
clientInputStream.close();
```

```
socketConnection.close();
```

```
}
```

```
}
```

```
import java.io.*;
import java.util.*;
```

Request serializable object

```
public class Request implements Serializable {

    private String opcode;
    private int num1, num2;

    Request(String op, int n1, int n2) {
        opcode = op;
        num1 = n1;
        num2 = n2;
    }

    public String getOpcode() {
        return opcode;
    }

    public int getFirst() {
        return num1;
    }

    public int getSecond() {
        return num2;
    }
}
```

```
import java.io.*;
import java.util.*;
```

Reply serializable object

```
public class Reply implements Serializable {
```

```
    private String opcode;
    private int num;
```

```
    public Reply() {
        opcode = null;
        num = 0;
    }
```

```
    public void setOpcode(String op) {
        opcode = op;
    }
```

```
    public void setValue(int n) {
        num = n;
    }
```

```
    public String getOpcode() {
        Return opcode;
    }
```

```
    public int getValue() {
        return num;
    }
```

```
}
```

Υλοποίηση υπολογισμού Server

```
import java.net.*;
import java.io.*;

public class ServerProtocol {

    private String opcode;
    private int num1, num2;
    private int tmp;
    private Reply res = new Reply();

    public Reply processRequest(Request theInput) {

        opcode = theInput.getOpcode();
        num1 = theInput.getFirst();
        num2 = theInput.getSecond();

        if (opcode.equals("ADD"))
        {res.setOpcode("OK"); tmp = num1+num2; res.setValue(tmp);}
        else if (opcode.equals("SUB"))
        {res.setOpcode("OK"); tmp = num1-num2; res.setValue(tmp);}
        else if (opcode.equals("MUL"))
        {res.setOpcode("OK"); tmp = num1*num2; res.setValue(tmp);}
        else if (opcode.equals("DIV"))
        {res.setOpcode("OK"); tmp = num1/num2; res.setValue(tmp);}
        else { res.setOpcode("ERR"); res.setValue(-9999); };
        return res;
    }
}
```



Κλήση απομακρυσμένης διαδικασίας

Προγραμματισμός υποδοχών: κριτική 1

- Ευέλικτη μέθοδος αλλά χαμηλού επιπέδου προγραμματισμός.
- Επικοινωνία σε επίπεδο bytes (συνήθως χαρακτήρων), επιτρέπει επικοινωνία μεταξύ εφαρμογών γραμμένων σε διαφορετικές γλώσσες που εκτελούνται σε διαφορετικά λειτουργικά συστήματα ή αρχιτεκτονικές*.
- Μέθοδος ανάπτυξης κυρίως βασικών δικτυακών εφαρμογών (ενδιάμεσου λογισμικού) ή παιχνιδιών για ταχύτητα.

* Υπάρχουν επεκτάσεις για επικοινωνία με δομές ή αντικείμενα αλλά χάνεται η καθολικότητα (πχ C/C++ struct ή Java serializable objects).

Προγραμματισμός υποδοχών: κριτική 2

- Απαιτεί λεπτομερή υλοποίηση πρωτοκόλλου εφαρμογής (πελάτη και διακομιστή).
- Τύποι δεδομένων και δομές δεδομένων για τη σύνταξη μηνυμάτων.
- Τύποι, ζεύγη, μηνυμάτων, κανόνες αιτήματος – απάντησης.
- Διαχείριση σφαλμάτων για όλα τα παραπάνω.

Προγραμματισμός υποδοχών: κριτική 3

- Απλό παράδειγμα: πρωτόκολλο για πρόσθεση δύο αριθμών. Αποστολή “add” “1053” “72381” και παραλαβή “ok” “82911” ή “err”.
- Θα μπορούσαμε να στείλουμε αριθμούς ή δομές δεδομένων (μηνύματα) που θα ορίσουμε (πχ serializable objects στη Java).
- Απώλεια της γενικότητας της λύσης: συγκεκριμένη γλώσσα προγραμματισμού, συγκεκριμένο (εικονικό) σύστημα χρόνου εκτέλεσης.
- Και πάλι θα πρέπει να συνταχθεί πρωτόκολλο, έστω και απλούστερο.

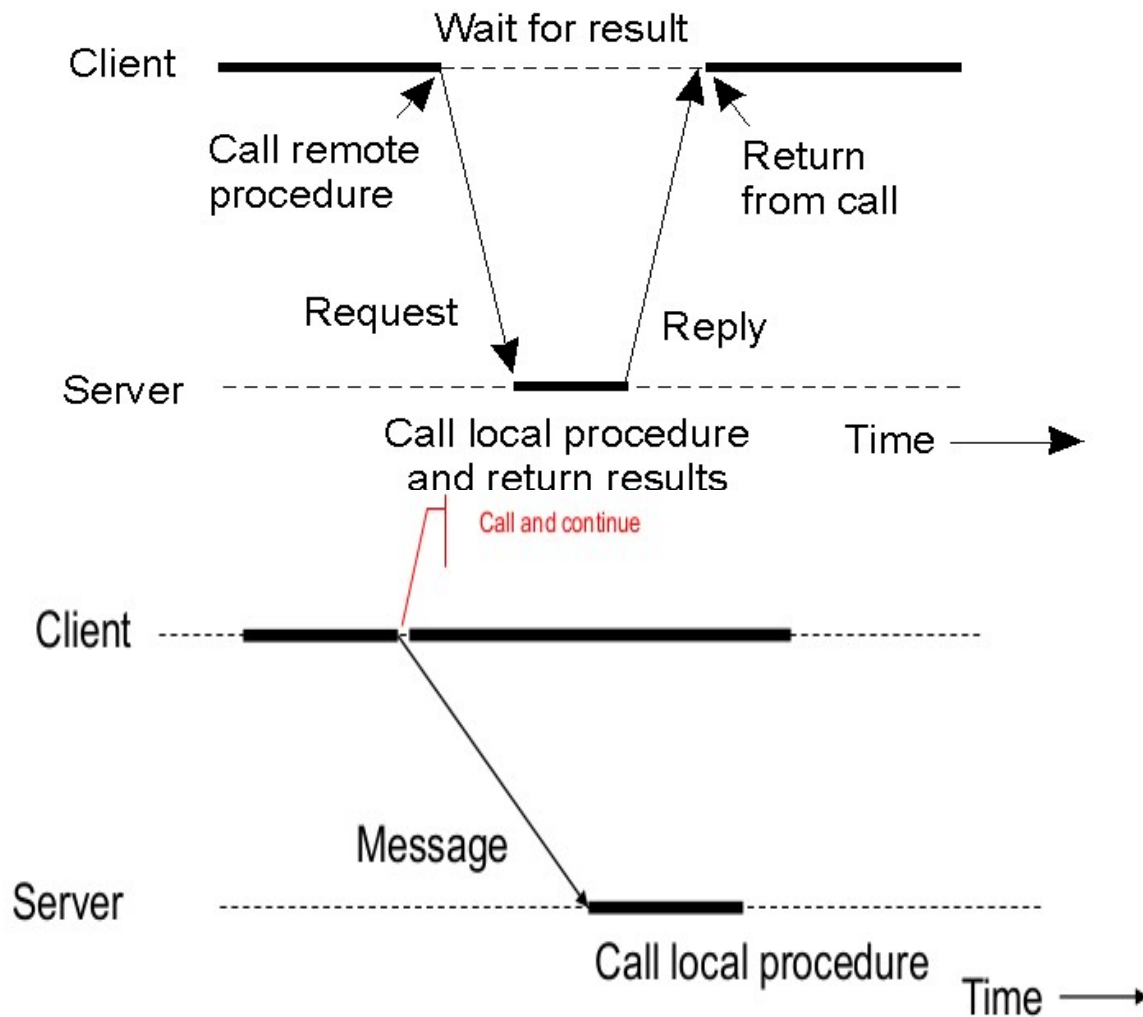
Προγραμματισμός υποδοχών: κριτική 4

- Μεγάλο μέρος του κώδικα παραμένει ίδιο στις τυπικές εφαρμογές: σύνδεση, εγκαθίδρυση επικοινωνίας, δημιουργία νημάτων στο διακομιστή, βασικός βρόχος διαλόγου.
- Θα μπορούσαμε να μετατρέψουμε τον κώδικα αυτό σε βιβλιοθήκη που να παράγεται “αυτόματα” ώστε να εστιάσουμε σε αυτό που μας ενδιαφέρει, δηλαδή το πρωτόκολλο εφαρμογής.

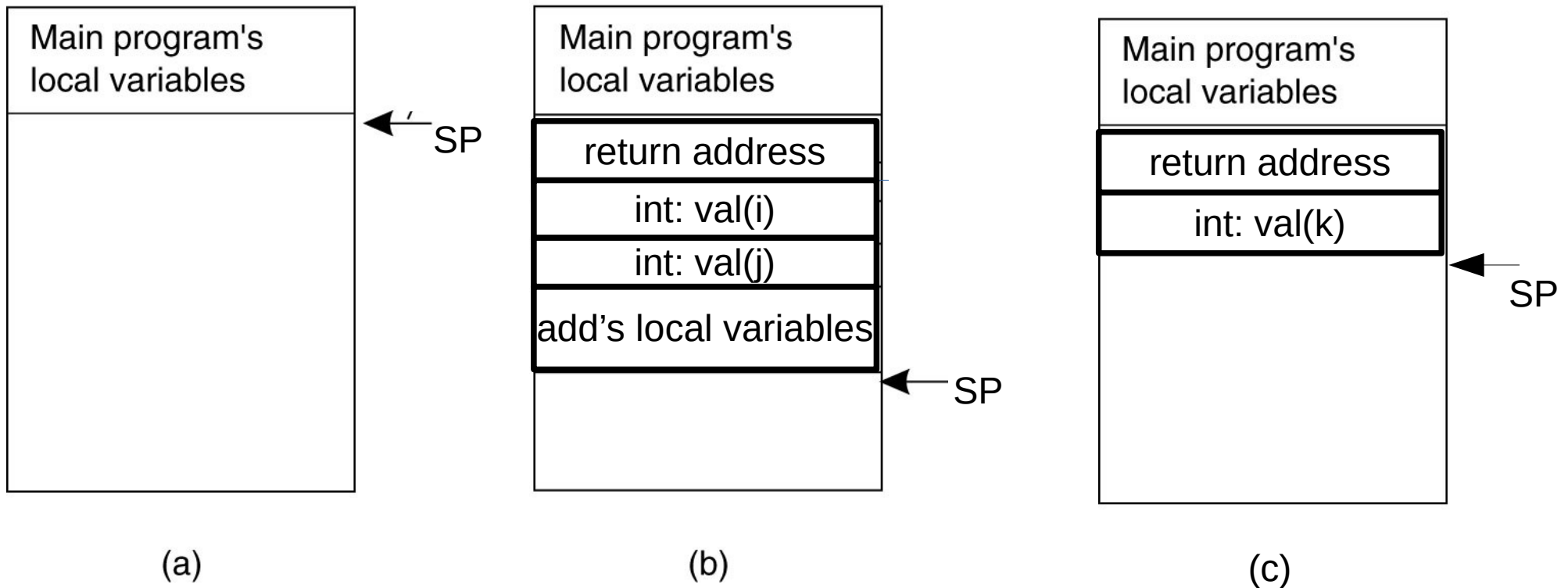
Μια εναλλακτική πρόταση: κλήση απομακρυσμένης διαδικασίας

- Διαμόρφωση της επικοινωνίας ως κλήση διαδικασίας (διαδικασία = ρουτίνα, συνάρτηση, μέθοδος..).
- Απόκρυψη της ρητής δημιουργίας, αποστολής παραλαβής μηνυμάτων.
- Απλό send – recn (μονόδρομο rendezvous) = απλή κλήση διαδικασίας, πχ `add (i, j)`.
- Αίτημα – απάντηση (αμφίδρομο rendezvous) = κλήση διαδικασίας με επιστροφή, πχ `k = add (i, j)`.
- Το πρωτόκολλο υλοποιείται με τις κλήσεις διαδικασιών.

Αμφίδρομη και μονόδρομη κλήση απομακρυσμένης διαδικασίας



Κλήση διαδικασίας στον ίδιο χώρο διευθύνσεων 1



Κλήση διαδικασίας `int k = add (int i, int j)` στον ίδιο χώρο διευθύνσεων.

(a) πριν τη κλήση (b) μετά τη κλήση (c) πριν την επιστροφή

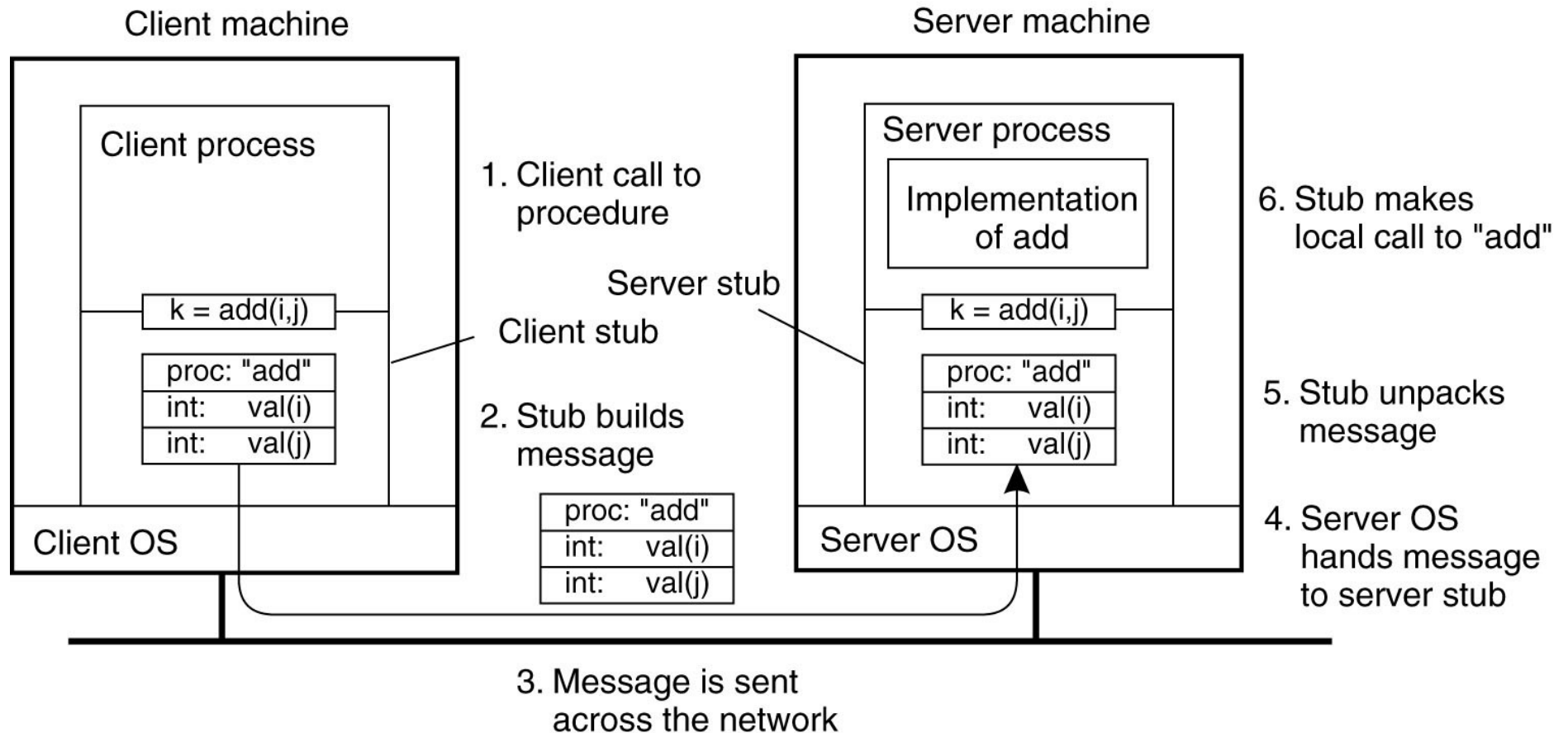
Main: η διαδικασία που καλεί (όχι απαραίτητα το κυρίως πρόγραμμα).

Κλήση απομακρυσμένης διαδικασίας

Remote procedure call (RPC) 1

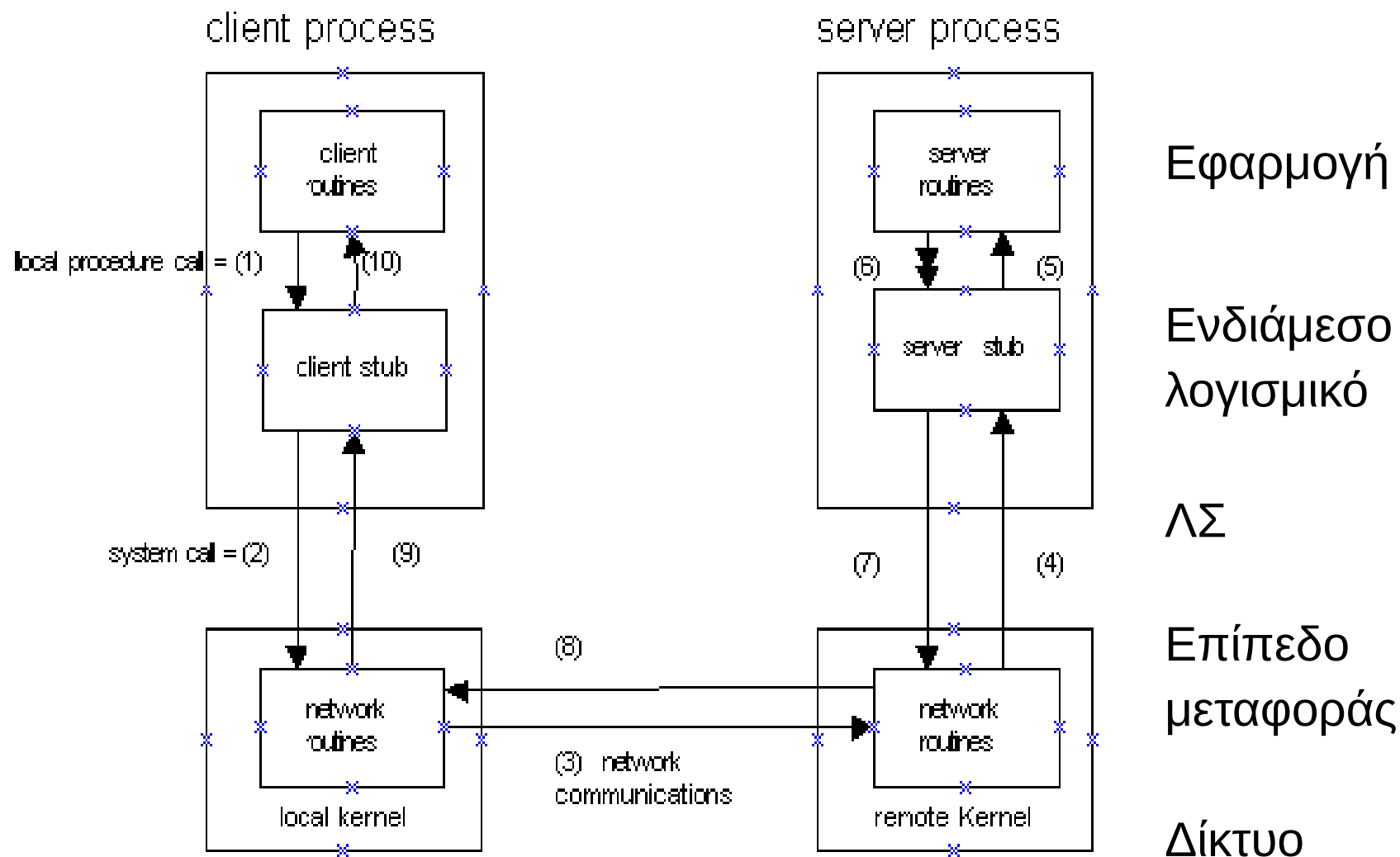
- Ο διακομιστής είναι πολυνηματικός / πολυδιεργασιακός, όχι ακολουθιακός ή με πολυπλεξία.
- Ο πελάτης και ο διακομιστής είναι διεργασίες που εκτελούνται σε διαφορετικούς χώρους διευθύνσεων: στο ίδιο ή διαφορετικό υπολογιστικό σύστημα (τοπική ή και δικτυακή επικοινωνία) και
- Στη γενική μορφή του RPC ο πελάτης και ο διακομιστής μπορεί να είναι υλοποιημένα στην ίδια ή διαφορετικές γλώσσες προγραμματισμού ή / και να εκτελούνται σε διαφορετική αρχιτεκτονική / διαφορετικό λειτουργικό σύστημα.

Κλήση απομακρυσμένης διαδικασίας 2



Φαίνεται μόνο η επικοινωνία της κλήσης `add(i,j)` , όχι της επιστροφής της τιμής `k`. Στο επόμενο σχήμα φαίνεται όλη η λειτουργία.

Κλήση απομακρυσμένης διαδικασίας 3



Γενική αρχιτεκτονική λογισμικού και λειτουργία μια κλήσης απομακρυσμένης διαδικασίας.

Κλήση απομακρυσμένης διαδικασίας 4

1. Ο πελάτης γνωρίζει την υπογραφή (διεπαφή) της add και συνθέτει την κλήση στη στοίβα του. Κατόπιν καλεί το στέλεχος του πελάτη (client stub).
2. Το στέλεχος του πελάτη συνθέτει το μήνυμα (marshalling, παράταξη) γνωρίζοντας τη διεύθυνση του διακομιστή. Καλεί το λειτουργικό σύστημα του πελάτη.
3. Το μήνυμα μεταβιβάζεται μέσω υποδοχών (τοπική ή δικτυακή επικοινωνία) από το λειτουργικό σύστημα του πελάτη στο λειτουργικό σύστημα του διακομιστή.
4. Το λειτουργικό σύστημα του διακομιστή μέσω της θύρας του μηνύματος αναγνωρίζει και καλεί το στέλεχος του διακομιστή (server stub).
5. Το στέλεχος του διακομιστή αποσυνθέτει το μήνυμα (unmarshalling, απο-παράταξη) και ενημερώνει το διακομιστή με τοπικό αίτημα.

Κλήση απομακρυσμένης διαδικασίας 5

6. Ο διακομιστής εκτελεί τη κλήση της τοπικής διαδικασίας add (υλοποίηση της add). Τοποθετεί το αποτέλεσμα στη στοίβα του και απαντά στο στέλεχος του διακομιστή.
7. Το στέλεχος του διακομιστή συνθέτει το μήνυμα γνωρίζοντας τη διεύθυνση του πελάτη από την αίτηση που έλαβε. Καλεί το λειτουργικό σύστημα του διακομιστή.
8. Το μήνυμα μεταβιβάζεται μέσω υποδοχών (τοπική ή δικτυακή επικοινωνία) από το λειτουργικό σύστημα του διακομιστή στο λειτουργικό σύστημα του πελάτη. Το λειτουργικό σύστημα του πελάτη μέσω της θύρας του μηνύματος αναγνωρίζει και καλεί το στέλεχος του πελάτη.
9. Το στέλεχος του πελάτη αποσυνθέτει το μήνυμα, τοποθετεί το αποτέλεσμα στη στοίβα κλήσεων και επιστρέφει στο πελάτη.
10. Ο πελάτης παραλαμβάνει το αποτέλεσμα από τη στοίβα του στην επιστροφή από τη κλήση απομακρυσμένης διαδικασίας.

Πλεονεκτήματα RPC

- Απόκρυψη λεπτομερειών της επικοινωνίας (υποδοχές, μηνύματα, ροές κλπ).
- Απλοποίηση πρωτοκόλλου εφαρμογής (τύποι δεδομένων, δομή μηνυμάτων, έλεγχος σφαλμάτων): όλος ο μηχανισμός βρίσκεται στις κλήσεις διαδικασιών.
- Διαφανής διαχείριση ταυτοχρονισμού: κάθε κλήση απομακρυσμένης διαδικασίας εκτελείται από διαφορετικό νήμα / διεργασία.
- (αλλά...) Η ταυτόχρονη εκτέλεση μπορεί να υλοποιείται είτε με απομονωμένες διεργασίες είτε με νήματα και να απαιτείται έλεγχος ταυτοχρονισμού.

Προβλήματα RPC

- Μεσολάβηση δικτύου (επικοινωνία δεδομένων): διαφανής οργάνωση τύπων δεδομένων και μορφών μηνυμάτων.
- Διαφορετικοί υπολογιστές, απομακρυσμένη αναφορά σε διαδικασία: όνομα διαδικασίας = διεύθυνση σε άλλο σύστημα / χώρο διευθύνσεων. Σύνδεση απομακρυσμένης αναφοράς με υποδοχή.
- (Πιθανώς) διαφορετικά: αρχιτεκτονική, λειτουργικό σύστημα, γλώσσα προγραμματισμού / σύστημα εκτέλεσης.